

THE PROMPT GUIDE

Building a Portfolio with AI

Every prompt I used to build my interactive portfolio site with Claude — the working ones, the dead ends, and the lessons in between. Reusable. Forkable. Honest about what didn't work.

AUTHOR

Arvind Singh

FORMAT

Reusable prompt sequence

STACK

React · TypeScript · Vite · Tailwind

USE THIS FOR

Personal portfolios, side projects, content posts

How to use this guide. Each prompt is a template. Replace the bracketed personalization fields and the JSON placeholder content with your own. The structure is what matters — the design, layout, and components stay the same. If you want to fork the result directly, duplicate this guide into your Notion workspace.

01 — What You'll Build

A single-page personal portfolio site with eight sections, a content layer decoupled into JSON, and a 3D-styled avatar in the hero. The site is built iteratively through a sequence of focused prompts — not one giant ask.

Sections

- **Hero.** Giant chrome-gradient name, centered avatar, social pill row.
- **About.** Bio paragraph bound to the JSON. No mid-word wrapping.
- **Experience.** Numbered 01/02/03 timeline of roles with quantified highlights.
- **Services.** Same numbered design — Backend, AI/LLM, Frontend, Cloud capabilities.
- **Projects.** Sticky dark cards. Highlighted project pinned first.
- **Testimonials.** Horizontal marquee, dark cards, decorative emojis.
- **Footer.** Three-column grid: brand, navigate, reach out.

Lessons before the prompts

Five things I learned the hard way.

- 1. Auto-tracing a photoreal 3D portrait into SVG produces 4,000+ paths and the LLM can't restructure it cleanly. Use a flat illustration style instead.*
- 2. Layered parallax with different easing per layer — pupils fast, eyes medium, head slow — is what makes character tracking feel alive. Whole-block movement looks like a security camera.*
- 3. Decoupling content into JSON saves you when the design pivots three times. Worth the hour of upfront work, every time.*
- 4. Force the AI to deliver in stages, not one giant code dump. The first stage catches the misinterpretations cheaply.*
- 5. Mid-word text wrapping (focu/s on, incredibl/e) comes from word-break: break-all. Always set word-break: normal explicitly on body paragraphs.*

02 — The Build Prompt

This is the master prompt. It scaffolds the entire project — every component, the data layer, the styling tokens. Paste it into Claude Code, Cursor, or any agentic coding tool with file-write access.

Tech stack

React 18 · TypeScript · Vite · Tailwind CSS · Framer Motion · lucide-react

Design tokens

- **Background:** #0C0C0C
- **Headline gradient:** linear-gradient(180deg, #646973 0%, #BBCCD7 100%)
- **Accent gradient:** purple → magenta → orange
- **Font family:** Kanit

The full prompt

```
Build a modern personal portfolio website for a senior software engineer.

Tech stack: React 18, TypeScript, Vite, Tailwind CSS, Framer Motion, lucide-react.

Design style:
- Dark background: #0C0C0C
- Large chrome/silver gradient display headlines
- Purple-magenta-orange gradient accent buttons
- Dark-on-dark cards with subtle gray borders
- Smooth animations, sticky project cards, horizontal marquee sections
- Kanit font family
- First screen is the actual portfolio hero – no landing-page fluff

Core requirements:
1. Vite + React + TypeScript app
2. Tailwind CSS configured
3. All content in src/data/portfolio.json
4. Typed usePortfolio() hook
5. No hardcoded profile/experience/project/testimonial content in components

Data structure (src/data/portfolio.json):
- profile: name, shortName, tagline, role, specialization, location,
  yearsOfExperience, bio, avatarSvg, social{github, instagram, linkedin,
  email, phone, website}
- skills.categories[]: name, items[]
- experience[]: company, role, period, location, summary, highlights[]
- projects[]: id, title, subtitle, description, stack[], role, year, link,
  image, highlight
- education[]
- testimonials[]: id, quote, name, role, avatarColor
```

Sections to build (in order):

1. HERO – full viewport, navbar, chrome-gradient "Hi, I'm {shortName}", centered avatar, tagline, social pill row, hide empty social links.
2. NAVBAR – HOME, ABOUT, SKILLS, PROJECTS, CONTACT, smooth scroll, scroll-margin-top: 80px on sections.
3. ABOUT – headline + bio paragraph from JSON, overflow-wrap: normal, word-break: normal (no mid-word breaks).
4. EXPERIENCE – numbered 01/02/03 rows from experience[], company-role title, period as monospace pill, summary, first 3 highlights, dividers.
5. SERVICES – same numbered design, hardcoded rows for Backend, AI/LLM, Frontend, Cloud (TODO comment to move to JSON later).
6. PROJECTS – sticky dark cards, highlight:true sorted first, numbered, LIVE PROJECT button hidden when link empty, image fallback to dark placeholder with title overlay.
7. TESTIMONIALS – dark background, decorative emojis (icons), CSS-only infinite horizontal marquee right-to-left, pause on hover, respect prefers-reduced-motion via scroll-snap fallback. Cards have italic quote, uppercase name, muted role, avatar circle with initial.
8. FOOTER – 3-col grid (stacked mobile), brand col with chrome-gradient name + specialization + location, NAVIGATE col with anchor links, REACH OUT col with email (copy button) + phone + SocialLinks. Bottom strip with divider, copyright left, build credit right.

Components:

```
src/components/Navbar.tsx
src/components/SocialLinks.tsx
src/components/HeroSection.tsx
src/components/AboutSection.tsx
src/components/ExperienceSection.tsx
src/components/ServicesSection.tsx
src/components/ProjectsSection.tsx
src/components/ProjectCard.tsx
src/components/TestimonialsSection.tsx
src/components/Footer.tsx
src/hooks/usePortfolio.ts
src/types/portfolio.ts
```

CSS:

```
.hero-heading {
background: linear-gradient(180deg, #646973 0%, #BBCCD7 100%);
-webkit-background-clip: text;
-webkit-text-fill-color: transparent;
background-clip: text;
}
html { scroll-behavior: smooth; }
+ testimonial marquee keyframes
+ reduced-motion media query
```

Quality:

- Fully responsive
- No broken links for empty URLs
- No mid-word wrapping in About
- npm run build succeeds
- README with install, scripts, structure, content-editing instructions

03 — The Avatar Prompt

The avatar is the visual signature of the site — the cursor-following character that bursts through the headline. Generate it with an image model, then background-remove and host it.

Customization fields

- **Pronouns.** Replace [He/His] with the subject's pronouns.
- **Skin / hair / eye color.** Main personalization points.
- **Hairstyle.** Quiff, curly, buzz cut, etc.
- **Facial hair.** Optional. Remove the beard/mustache line if not applicable.
- **Accessories.** Earrings, glasses — add or remove freely.
- **Background.** Keep deep black — it makes the character pop on a dark site.

The prompt (template)

A high-quality, professional 3D animated head-and-shoulders portrait, rendered in a polished, Pixar-style aesthetic. The character has smooth, matte skin [with faint freckles across the nose and cheeks], and very large, expressive [dark brown] eyes. [He] has a warm, happy smile showing neat white teeth. [His] hair is highly detailed, sculpted, voluminous, and [dark black, styled in a swept-up quiff]. [He has a well-groomed dark beard and mustache, and thick, arched eyebrows.] [He is wearing small, polished gold hoop earrings in both ears.] The lighting is dramatic, high-contrast studio lighting, casting soft shadows. The background is a solid, deep black, ensuring the character pops.

***Don't skip the keyword "Pixar-style aesthetic".** Alternatives like "3D render" or "stylized" produce inconsistent results — sometimes photoreal, sometimes cartoon, sometimes uncanny. The Pixar phrasing anchors the model to a specific, replicable style.*

04 — Iteration Prompts

After the initial scaffold, three follow-up prompts handle the most common issues: the headline that overflows, the avatar that won't center, and the content sections that need to be wired to the JSON.

01

Fix the hero headline so it never clips

Use `font-size: clamp(3rem, 13vw, 14rem)` on the headline, `white-space: nowrap` on the text, and `overflow: hidden` on the hero container as a safety net. Validate at 375px, 768px, 1280px, and 1920px viewports.

02

Replace absolute-positioned avatar with CSS Grid

The hero becomes a grid with rows for nav, headline, avatar, and tagline. The avatar spans grid rows 2 and 3 with z-index above the headline — this creates the "character bursting through the title" effect. Cleaner than absolute positioning and consistent across breakpoints.

03

Wire each section to `portfolio.json` via `usePortfolio()`

Build each section component to read from the typed hook — never hardcode names, dates, or copy. When the design pivots (and it will), only the JSON changes. The components stay untouched.

***The single most useful instruction.** Add this line to every long prompt: "Show me the result of step 1 before continuing. Confirm with me before building the rest." It catches misinterpretations cheaply and stops the AI from generating 800 lines of code in the wrong direction.*

05 — The Data Layer

The single most important architectural decision in the project: every piece of content lives in `src/data/portfolio.json`. Components import a typed hook, never strings. When you change jobs, ship a new project, or add testimonials, you edit one file.

```

{
  "profile": {
    "name": "Your Name",
    "shortName": "First name for the headline",
    "tagline": "One line that answers 'who are you?'",
    "role": "Your title",
    "specialization": "Domain · Domain · Domain",
    "location": "City, Country",
    "yearsOfExperience": "4+",
    "bio": "Two to three sentences. Lead with what you build.",
    "avatarSvg": "/src/assets/img/avatar.svg",
    "social": {
      "github": "https://github.com/...",
      "linkedin": "https://linkedin.com/in/...",
      "instagram": "",
      "email": "you@domain.com",
      "phone": "+00 0000000000",
      "website": "https://yoursite.com"
    }
  },
  "experience": [
    {
      "company": "Company Name",
      "role": "Senior Whatever Engineer",
      "period": "2024 – Present",
      "location": "Remote / City",
      "summary": "One sentence framing the role.",
      "highlights": [
        "Quantified impact bullet – what you shipped, what changed.",
        "Quantified impact bullet – what you shipped, what changed.",
        "Quantified impact bullet – what you shipped, what changed."
      ]
    }
  ],
  "projects": [
    {
      "id": "project-slug",
      "title": "Project Name",
      "subtitle": "One-line tagline",
      "description": "Two sentences. What it does, what's interesting.",
      "stack": ["React", "TypeScript", "Whatever"],
      "role": "Your role on it",
      "year": "2026",
      "link": "https://...",
      "image": "/path/to/screenshot.png",
      "highlight": true
    }
  ],
  "testimonials": [
    {
      "id": "t1",
      "quote": "Short, specific, in their voice.",
      "name": "Person Name",
      "role": "Role, Company",
      "avatarColor": "#A855F7"
    }
  ]
}

```

Tip on testimonials. If you don't have real ones yet, leave the array empty rather than filling with obviously-fake quotes. The marquee section can be hidden conditionally when `testimonials.length === 0`. Fake testimonials are detected instantly and damage credibility.

06 — What Didn't Work

Honesty section. These are the dead ends I hit. They cost me hours each. If you skip them, you'll save the time.

01

Vectorizing a 3D portrait into SVG

I generated a Pixar-style portrait and tried to convert it to a layered SVG so I could animate the eyes independently. The auto-tracer produced 4,244 paths — every shadow gradient on the skin became its own path. The LLM couldn't reliably identify which paths were eyes versus nostril shadows. Lesson: photoreal portraits don't vectorize cleanly.

02

Trying to make AI "redesign" a generic template

I started with a generic 3D-designer portfolio template and asked the AI to adapt it for a backend engineer. The result kept smuggling 3D-modeling references into the copy. Lesson: it's faster to delete the template entirely and rebuild from a JSON-first prompt than to negotiate with a misaligned base.

03

Skipping the staged-delivery instruction

Early prompts asked for everything in one response. The AI generated 800+ lines of code with assumptions I would have caught in 30 seconds during a staged review. Lesson: long prompts should always end with "deliver in stages, confirm at each step."

***The meta-lesson.** AI coding tools are fast at execution but bad at saying "this whole approach won't work." That's still your job. The two hours I lost on SVG vectorization could have been a 30-second "is this the right approach?" question to a human or a more skeptical model. Use AI for execution, not for strategic validation.*

07 — Make It Yours

Take the prompts. Change the names. Swap the design tokens. Ship something. The fastest portfolio is the one you actually finish — and a JSON-first architecture means you can finish v1 in a weekend and keep editing for years without touching components.

If something in this guide saved you time

Share it forward. The best signal that a prompt guide is useful is that it shows up in someone else's project. No attribution required, no permission needed.

BUILT WITH

Claude · React · TypeScript · Tailwind

DESIGNED FOR

Anyone shipping a portfolio in 2026